
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 10

Estruturas, Uniões, Manipulações de Bits e Enumerações
em C

Exercícios (10.5–10.20)

Resolvidos passo a passo, com código completo

Contents

1	Exercício 10.5 – Definir Estruturas e Uniões	2
2	Exercício 10.6 – Acesso a Membros	4
3	Exercício 10.7 – Embaralhamento Eficiente de Cartas	4
4	Exercício 10.8 – União integer	7
5	Exercício 10.9 – União pontoFlutuante	9
6	Exercício 10.10 – Deslocamento à Direita	10
7	Exercício 10.11 – displayBits para 2 Bytes	12
8	Exercício 10.12 – Multiplicação por Deslocamento	12
9	Exercício 10.13 – Compactar 2 Caracteres	14
10	Exercício 10.14 – Descompactar 2 Caracteres	16
11	Exercício 10.15 – Compactar 4 Caracteres	17
12	Exercício 10.16 – Descompactar 4 Caracteres	18
13	Exercício 10.17 – Inverter Bits	19
14	Exercício 10.18 – displayBits Portátil	20
15	Exercício 10.19 – Qual é o valor de X?	22
16	Exercício 10.20 – O que mystery faz?	22

1. Exercício 10.5 – Definir Estruturas e Uniões

Resposta detalhada

a) Estrutura estoque:

```
1 struct estoque {  
2     char nomePeca[ 30 ];  
3     int  numPeca;  
4     float preco;  
5     int  quantidade;  
6     int  pedido;  
7 };
```

b) União dados:

```
1 union dados {  
2     char  c;  
3     short s;  
4     long  b;  
5     float f;  
6     double d;  
7 };
```

c) Estrutura endereco:

```
1 struct endereco {  
2     char rua[ 25 ];  
3     char cidade[ 20 ];  
4     char estado[ 2 ];  
5     char cep[ 8 ];  
6 };
```

d) Estrutura aluno aninhando endereco:

```
1 struct aluno {  
2     char nome[ 15 ];  
3     char sobrenome[ 15 ];  
4     struct endereco endResid; /* estrutura  
5                               aninhada */  
6 };
```

e) Estrutura teste com 16 campos de bit:

```
1 struct teste {  
2     unsigned int a : 1;  
3     unsigned int b : 1;  
4     unsigned int c : 1;  
5     unsigned int d : 1;  
6     unsigned int e : 1;  
7     unsigned int f : 1;  
8     unsigned int g : 1;  
9     unsigned int h : 1;  
10    unsigned int i : 1;
```

```
11     unsigned int j : 1;  
12     unsigned int k : 1;  
13     unsigned int l : 1;  
14     unsigned int m : 1;  
15     unsigned int n : 1;  
16     unsigned int o : 1;  
17     unsigned int p : 1;  
18 };
```

Campos de bit permitem economizar memória quando precisamos apenas de poucos bits para representar valores – útil em estruturas que mapeiam registradores de hardware ou protocolos de rede.

2. Exercício 10.6 – Acesso a Membros

Resposta detalhada

Dadas as declarações da estrutura aninhada `cliente` e o ponteiro `ptrCliente = ®Cliente`:

- a) `regCliente.sobrenome`
- b) `ptrCliente->sobrenome`
- c) `regCliente.nome`
- d) `ptrCliente->nome`
- e) `regCliente.numCliente`
- f) `ptrCliente->numCliente`
- g) `regCliente.pessoal.telefone`
- h) `ptrCliente->pessoal.telefone`
- i) `regCliente.pessoal.endereco`
- j) `ptrCliente->pessoal.endereco`
- k) `regCliente.pessoal.cidade`
- l) `ptrCliente->pessoal.cidade`
- m) `regCliente.pessoal.estado`
- n) `ptrCliente->pessoal.estado`
- o) `regCliente.pessoal.cep`
- p) `ptrCliente->pessoal.cep`

Boa prática de programação

Padrão de acesso:

- Variável simples: `var.membro`
- Via ponteiro: `ptr->membro` (equivale a `(*ptr).membro`)
- Aninhada: `ptr->grupo.membro` (acompanhe a hierarquia)

3. Exercício 10.7 – Embaralhamento Eficiente de Cartas

Resposta detalhada

```
1  /* Ex10.7: cartas_struct.c
2     Embaralhamento e distribuicao usando struct Card */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <time.h>
6  #define CARTAS 52
7
8  struct card {
9      const char *face;
10     const char *suit;
11 };
12
13 typedef struct card Card;
14
```

```
15 void fillDeck( Card * const baralho,
16               const char * faces[],
17               const char * suits[] );
18 void shuffleDeck( Card * const baralho ); /* Fisher-
19      Yates */
20 void dealDeck( const Card * const baralho );
21
22 int main( void )
23 {
24     /* Inicializa arrays de nomes */
25     const char *faces[] = {
26         "As", "Dois", "Tres", "Quatro", "Cinco", "Seis", "
27         Sete",
28         "Oito", "Nove", "Dez", "Valete", "Dama", "Rei"
29     };
30     const char *suits[] = { "Copas", "Ouros", "Espadas",
31         "Paus" };
32
33     Card deck[ CARTAS ];
34
35     srand( time( NULL ) );
36
37     fillDeck( deck, faces, suits );
38     shuffleDeck( deck );
39     dealDeck( deck );
40
41     return 0;
42 }
43
44 void fillDeck( Card * const baralho,
45               const char * faces[],
46               const char * suits[] )
47 {
48     int i;
49     for ( i = 0; i < CARTAS; ++i ) {
50         baralho[ i ].face = faces[ i % 13 ];
51         baralho[ i ].suit = suits[ i / 13 ];
52     }
53 }
54
55 /* Fisher-Yates: O(n), sem adiamento indefinido */
56 void shuffleDeck( Card * const baralho )
57 {
58     int i, j;
59     Card temp;
60     for ( i = CARTAS - 1; i > 0; --i ) {
61         j = rand() % ( i + 1 );
62         temp = baralho[ i ];
63         baralho[ i ] = baralho[ j ];
64         baralho[ j ] = temp;
```

```
62     }
63 }
64
65 /* Distribui em formato de 2 colunas */
66 void dealDeck( const Card * const baralho )
67 {
68     int i;
69     for ( i = 0; i < CARTAS; ++i ) {
70         printf( "%-7s de %-10s%s",
71                 baralho[ i ].face,
72                 baralho[ i ].suit,
73                 ( i % 2 == 0 ) ? "    " : "\n" );
74     }
75 }
```

Pontos-chave:

- `struct card` agrupa `face` e `suit` – duas informações por carta.
- Fisher-Yates elimina o adiamento indefinido do algoritmo ingênuo.
- `Card * const baralho` – ponteiro constante (não pode mudar para onde aponta) mas o conteúdo apontado pode mudar.

4. Exercício 10.8 – União integer

Resposta detalhada

```
1  /* Ex10.8: union_integer.c
2     Demonstra como diferentes tipos compartilham
3     memoria */
4
5  #include <stdio.h>
6
7  union integer {
8      char c;
9      short s;
10     int i;
11     long b;
12 };
13
14 int main( void )
15 {
16     union integer u;
17
18     /* Aceita um char */
19     printf( "Digite um char: " );
20     scanf( " %c", &u.c );
21     printf( "\nApos atribuir char:\n" );
22     printf( "  c = %c\n", u.c );
23     printf( "  s = %hd\n", u.s );
24     printf( "  i = %d\n", u.i );
25     printf( "  b = %ld\n", u.b );
26
27     /* Aceita um short */
28     printf( "\nDigite um short: " );
29     scanf( "%hd", &u.s );
30     printf( "Apos atribuir short:\n" );
31     printf( "  c = %c (codigo %d)\n", u.c, u.c );
32     printf( "  s = %hd\n", u.s );
33     printf( "  i = %d\n", u.i );
34     printf( "  b = %ld\n", u.b );
35
36     /* Aceita int e long */
37     printf( "\nDigite um int: " );
38     scanf( "%d", &u.i );
39     printf( "Apos atribuir int:\n" );
40     printf( "  c = %c (codigo %d)\n", u.c, u.c );
41     printf( "  s = %hd\n", u.s );
42     printf( "  i = %d\n", u.i );
43     printf( "  b = %ld\n", u.b );
44
45     printf( "\nDigite um long: " );
46     scanf( "%ld", &u.b );
47     printf( "Apos atribuir long:\n" );
```



```
46     printf( "    c = %c (codigo %d)\n", u.c, u.c );
47     printf( "    s = %hd\n", u.s );
48     printf( "    i = %d\n", u.i );
49     printf( "    b = %ld\n", u.b );
50
51     return 0;
52 }
```

Os valores sempre são impressos corretamente? NÃO! Como todos os membros *compartilham* memória, atribuir a um membro destrói o conteúdo dos outros. Apenas o último membro atribuído é confiável; os demais mostram “lixo” do ponto de vista lógico (na verdade, são reinterpretações dos mesmos bits sob outros tipos).

Boa prática de programação

Unões são úteis quando uma variável precisa ser interpretada de várias formas, ou quando queremos economizar memória sabendo que apenas *um* dos campos é válido em cada momento. Em geral, mantenha um “tag” separado indicando qual membro está atualmente válido.

5. Exercício 10.9 – União pontoFlutuante

Resposta detalhada

```
1  /* Ex10.9: union_float.c */
2  #include <stdio.h>
3
4  union pontoFlutuante {
5      float      f;
6      double     d;
7      long double x;
8  };
9
10 int main( void )
11 {
12     union pontoFlutuante u;
13
14     printf( "Digite um float: " );
15     scanf( "%f", &u.f );
16     printf( "    f=%f    d=%f    x=%Lf\n\n", u.f, u.d, u.x )
17         ;
18
19     printf( "Digite um double: " );
20     scanf( "%lf", &u.d );
21     printf( "    f=%f    d=%f    x=%Lf\n\n", u.f, u.d, u.x )
22         ;
23
24     printf( "Digite um long double: " );
25     scanf( "%Lf", &u.x );
26     printf( "    f=%f    d=%f    x=%Lf\n", u.f, u.d, u.x );
27
28     return 0;
29 }
```

Mesma conclusão do Ex. 10.8: apenas o último valor atribuído faz sentido. A diferença aqui é que os tipos `float` (4 bytes), `double` (8 bytes) e `long double` (10–16 bytes, dependendo do sistema) têm representação IEEE 754 muito diferentes – interpretar os mesmos bits como tipos diferentes produz lixo numérico.

Tamanho da união: a união ocupa o tamanho do *maior* membro – `sizeof(long double)` no caso.

6. Exercício 10.10 – Deslocamento à Direita

Resposta detalhada

```
1  /* Ex10.10: shift_direita.c */
2  #include <stdio.h>
3
4  void displayBits( unsigned valor );
5
6  int main( void )
7  {
8      unsigned valor;
9
10     printf( "Digite um inteiro nao negativo: " );
11     scanf( "%u", &valor );
12
13     printf( "\nAntes do deslocamento:  " );
14     displayBits( valor );
15
16     valor >>= 4;  /* desloca 4 bits a direita */
17
18     printf( "Apos >>= 4:                " );
19     displayBits( valor );
20
21     return 0;
22 }
23
24 void displayBits( unsigned valor )
25 {
26     unsigned i;
27     unsigned mask = 1u << ( sizeof( unsigned ) * 8 - 1
28                             );
29
30     for ( i = 1; i <= sizeof( unsigned ) * 8; ++i ) {
31         putchar( valor & mask ? '1' : '0' );
32         valor <<= 1;
33
34         if ( i % 8 == 0 ) putchar( ' ' );
35     }
36     putchar( '\n' );
37 }
```

O que entra nos bits vagos?

- Para unsigned: sempre 0s. Esse é chamado **deslocamento lógico**.
- Para int (signed) negativo: depende da implementação – a maioria usa **deslocamento aritmético**, copiando o bit de sinal (1). Em compiladores modernos da família GCC, signed right shift de negativo preserva o sinal.

Boa prática de programação

Para resultados portáteis e previsíveis, prefira sempre tipos **unsigned** ao realizar manipulações de bits.

7. Exercício 10.11 – displayBits para 2 Bytes

Resposta detalhada

A função `displayBits` do Ex. 10.10 já é portátil: ela usa `sizeof(unsigned) * 8` para determinar o número de bits. Em sistemas com `unsigned` de 2 bytes (16 bits), o loop itera 16 vezes; com 4 bytes, 32 vezes.

Versão original do livro (não portátil, fixa em 4 bytes):

```
1  /* Versao fixa em 32 bits */
2  void displayBits( unsigned valor )
3  {
4      int i;
5      unsigned mask = 1 << 31;  /* fixo em 32 bits */
6
7      for ( i = 1; i <= 32; ++i ) {
8          putchar( valor & mask ? '1' : '0' );
9          valor <<= 1;
10         if ( i % 8 == 0 ) putchar( ' ' );
11     }
12     putchar( '\n' );
13 }
```

Para 2 bytes, basta mudar `31` → `15` e `32` → `16`.

8. Exercício 10.12 – Multiplicação por Deslocamento

Resposta detalhada

Fato: deslocar à esquerda em 1 bit equivale a multiplicar por 2. Em n bits, equivale a multiplicar por 2^n .

```
1  /* Ex10.12: potencia_dois.c */
2  #include <stdio.h>
3
4  void displayBits( unsigned valor );
5
6  unsigned potencia2( unsigned numero, unsigned pot )
7  {
8      return numero << pot;
9  }
10
11 int main( void )
12 {
13     unsigned numero, pot;
14
15     printf( "Digite numero e expoente: " );
16     scanf( "%u%u", &numero, &pot );
17
18     unsigned resultado = potencia2( numero, pot );
```

```
19
20     printf( "\n%u * 2^%u = %u\n\n", numero, pot,
21             resultado );
22     printf( "Bits de %u: ", numero );      displayBits(
23             numero );
24     printf( "Bits de resultado: " );      displayBits(
25             resultado );
26
27     return 0;
28 }
```

Por que funciona? A representação binária “empurra” todos os bits para posições mais altas. Como cada posição vale 2^k , mover um bit uma posição à esquerda multiplica seu peso por 2.

Exemplo: $5 = 101_2$. $5 \ll 2 = 10100_2 = 20 = 5 \times 4 = 5 \times 2^2$.

9. Exercício 10.13 – Compactar 2 Caracteres

Resposta detalhada

```

1  /* Ex10.13: compactar.c */
2  #include <stdio.h>
3
4  void displayBits( unsigned valor );
5  unsigned compactaCaracteres( char a, char b );
6
7  int main( void )
8  {
9      char a, b;
10
11     printf( "Digite dois caracteres: " );
12     scanf( " %c %c", &a, &b );
13
14     printf( "\nCaractere 1 ('%c'): ", a );
15     displayBits( ( unsigned )a );
16
17     printf( "Caractere 2 ('%c'): ", b );
18     displayBits( ( unsigned )b );
19
20     unsigned compactado = compactaCaracteres( a, b );
21
22     printf( "\nCompactado:          " );
23     displayBits( compactado );
24
25     return 0;
26 }
27
28 unsigned compactaCaracteres( char a, char b )
29 {
30     unsigned resultado;
31
32     resultado = a;           /* coloca 'a' nos 8
33                               bits baixos */
34     resultado <= 8;          /* desloca para os 8
35                               bits altos */
36     resultado |= ( unsigned )b; /* coloca 'b' nos 8
37                               bits baixos */
38
39     return resultado;
40 }

```

Exemplo: para 'A' (65 = 01000001) e 'B' (66 = 01000010):

a) resultado = 0x41 = 00000000 01000001

b) resultado <= 8 → 01000001 00000000

c) resultado |= 0x42 → 01000001 01000010

Os dois caracteres agora ocupam 16 bits da variável unsigned, com 'A' nos bits altos e 'B' nos bits baixos.

Por que isso é útil? Em sistemas embarcados e protocolos de rede, é comum *empacotar* múltiplos valores pequenos em uma variável maior para economizar memória ou largura de banda.

10. Exercício 10.14 – Descompactar 2 Caracteres

Resposta detalhada

```
1  /* Ex10.14: descompactar.c */
2  #include <stdio.h>
3
4  void displayBits( unsigned valor );
5
6  void descompactaCaracteres( unsigned valor, char *a,
7                             char *b )
8  {
9      /* Mascara 65280 = 0x0000FF00 isola os bits 8-15
10         */
11      *a = ( valor & 65280 ) >> 8;
12
13      /* Mascara 255 = 0x000000FF isola os bits 0-7 */
14      *b = valor & 255;
15  }
16
17 int main( void )
18 {
19     unsigned valor;
20     char a, b;
21
22     printf( "Digite um inteiro compactado: " );
23     scanf( "%u", &valor );
24
25     printf( "\nValor compactado: " );
26     displayBits( valor );
27
28     descompactaCaracteres( valor, &a, &b );
29
30     printf( "\nCaractere 1 ('%c'): ", a );
31     displayBits( ( unsigned )a );
32
33     printf( "Caractere 2 ('%c'): ", b );
34     displayBits( ( unsigned )b );
35
36     return 0;
37 }
```

Técnica das máscaras:

- 0xFF00 (65280) tem apenas os bits 8–15 acesos.
- AND com essa máscara *isola* apenas esses bits.
- » 8 desloca para a posição correta de `char`.
- 0x00FF (255) isola os bits 0–7 sem necessidade de deslocar.

11. Exercício 10.15 – Compactar 4 Caracteres

Resposta detalhada

```
1  /* Ex10.15: compactar4.c */
2  #include <stdio.h>
3
4  unsigned compactaCaracteres4( char a, char b, char c,
5                                char d )
6  {
7      unsigned resultado = 0;
8
9      resultado = ( unsigned )a;  /* a nos bits 24-31
10     */
11     resultado <= 8;
12     resultado |= ( unsigned )b;  /* b nos bits 16-23
13     */
14     resultado <= 8;
15     resultado |= ( unsigned )c;  /* c nos bits 8-15 */
16     resultado <= 8;
17     resultado |= ( unsigned )d;  /* d nos bits 0-7 */
18
19     return resultado;
20 }
21
22 int main( void )
23 {
24     char a, b, c, d;
25     printf( "Digite 4 caracteres: " );
26     scanf( " %c %c %c %c", &a, &b, &c, &d );
27
28     unsigned compactado = compactaCaracteres4( a, b, c,
29                                                d );
30     printf( "Compactado: 0x%08X\n", compactado );
31
32     return 0;
33 }
```

Exemplo: “ABCD” empacotado fica 0x41424344. Este é o conceito por trás da representação de inteiros de 4 bytes em arquivos – e da forma como sistemas big-endian/little-endian organizam dados na memória.

12. Exercício 10.16 – Descompactar 4 Caracteres

Resposta detalhada

```
1  /* Ex10.16: descompactar4.c */
2  #include <stdio.h>
3
4  void descompactaCaracteres4( unsigned v,
5                               char *a, char *b, char *c
6                               , char *d )
7  {
8      unsigned mascara = 255; /* 0x000000FF */
9
10     *d = v & mascara;
11     *c = ( v >> 8 ) & mascara;
12     *b = ( v >> 16 ) & mascara;
13     *a = ( v >> 24 ) & mascara;
14 }
15
16 int main( void )
17 {
18     unsigned valor;
19     char a, b, c, d;
20
21     printf( "Digite valor compactado (em hex, ex: 0
22             x41424344): " );
23     scanf( "%x", &valor );
24
25     descompactaCaracteres4( valor, &a, &b, &c, &d );
26     printf( "Caracteres: %c %c %c %c\n", a, b, c, d );
27
28     return 0;
29 }
```

Alternativa: máscaras separadas (técnica do enunciado):

```
1  unsigned mask_d = 255u; /* bits 0-7 */
2  unsigned mask_c = 255u << 8; /* bits 8-15 */
3  unsigned mask_b = 255u << 16; /* bits 16-23 */
4  unsigned mask_a = 255u << 24; /* bits 24-31 */
5
6  *d = v & mask_d;
7  *c = ( v & mask_c ) >> 8;
8  *b = ( v & mask_b ) >> 16;
9  *a = ( v & mask_a ) >> 24;
```

As duas abordagens são equivalentes; a primeira (deslocar antes de mascarar) é ligeiramente mais eficiente.

13. Exercício 10.17 – Inverter Bits

Resposta detalhada

```
1  /* Ex10.17: reverseBits.c */
2  #include <stdio.h>
3
4  void displayBits( unsigned valor );
5
6  unsigned reverseBits( unsigned valor )
7  {
8      unsigned resultado = 0;
9      unsigned numBits = sizeof( unsigned ) * 8;
10     unsigned i;
11
12     for ( i = 0; i < numBits; ++i ) {
13         /* Pega o bit menos significativo de valor */
14         unsigned bit = valor & 1;
15
16         /* Desloca resultado para a esquerda e
17            adiciona o bit */
18         resultado = ( resultado << 1 ) | bit;
19
20         /* Desloca valor para o proximo bit */
21         valor >>= 1;
22     }
23
24     return resultado;
25 }
26
27 int main( void )
28 {
29     unsigned v;
30
31     printf( "Digite um unsigned: " );
32     scanf( "%u", &v );
33
34     printf( "\nAntes:  " ); displayBits( v );
35
36     unsigned inv = reverseBits( v );
37     printf( "Apos:    " ); displayBits( inv );
38
39     return 0;
40 }
```

Lógica: percorremos cada bit de valor (do menos para o mais significativo) e o colocamos como bit mais significativo de resultado. É como “empilhar” bits ao contrário.

Aplicações: processamento de sinais digitais, FFT (*Fast Fourier Transform*), algoritmos de checksum.

14. Exercício 10.18 – displayBits Portátil

Resposta detalhada

```
1  /* Ex10.18: displayBits_portavel.c
2     Funciona em sistemas com unsigned de qualquer
3     tamanho */
4
5  #include <stdio.h>
6
7  void displayBits( unsigned valor )
8  {
9     /* sizeof(unsigned) * 8 = numero de bits da
10     arquitetura */
11     unsigned numBits = sizeof( unsigned ) * 8;
12     unsigned mask = 1u << ( numBits - 1 );
13     unsigned i;
14
15     printf( "%-10u = ", valor );
16
17     for ( i = 1; i <= numBits; ++i ) {
18         putchar( valor & mask ? '1' : '0' );
19         valor <<= 1;
20         if ( i % 8 == 0 ) putchar( ' ' );
21     }
22     putchar( '\n' );
23 }
24
25 int main( void )
26 {
27     printf( "Tamanho de unsigned: %zu bytes (%zu bits)
28     \n",
29     sizeof( unsigned ), sizeof( unsigned ) * 8
30     );
31
32     displayBits( 0u );
33     displayBits( 1u );
34     displayBits( 255u );
35     displayBits( 65535u );
36     displayBits( ~0u ); /* todos os bits 1 */
37
38     return 0;
39 }
```

Chave da portabilidade: `sizeof` é avaliado em tempo de compilação e retorna o tamanho exato do tipo na arquitetura atual. Em qualquer sistema (2, 4, ou 8 bytes), a função funciona sem modificações.

Boa prática de programação

Sempre que escrever código que dependa do tamanho de tipos, use `sizeof` – nunca constantes literais como 4, 32 ou 64. Seu código será automaticamente portátil entre arquiteturas.

15. Exercício 10.19 – Qual é o valor de X?

Resposta detalhada

Função suspeita:

```
1  int multiple( int num )
2  {
3      int i, mask = 1, mult = 1;
4      for ( i = 1; i <= 10; i++, mask <= 1 ) {
5          if ( ( num & mask ) != 0 ) {
6              mult = 0;
7              break;
8          }
9      }
10     return mult;
11 }
```

Análise passo a passo:

A função testa os 10 bits menos significativos de `num`. Se *qualquer um* dos primeiros 10 bits for 1, retorna 0 (não é múltiplo). Caso contrário, retorna 1.

Quando os 10 bits menos significativos são todos zero? Quando `num` é múltiplo de $2^{10} = 1024$.

Verificação:

- `num = 1024 = 210`: binário 10000000000 (apenas o 11º bit é 1). Os bits 0–9 são todos 0. Retorna 1.
- `num = 1025`: bit 0 é 1. Retorna 0.
- `num = 2048`: 100000000000 (12º bit é 1, bits 0–9 são 0). Retorna 1.

Resposta: $X = 1024$ (2^{10}).

A função determina se `num` é múltiplo de 1024.

16. Exercício 10.20 – O que `mystery` faz?

Resposta detalhada

Função misteriosa:

```
1  int mystery( unsigned bits )
2  {
3      unsigned i, total = 0;
4      unsigned mask = 1 << 31;
5
6      for ( i = 1; i <= 32; i++, bits <= 1 ) {
7          if ( ( bits & mask ) == mask ) {
8              total++;
9          }
10     }
11     return !( total % 2 ) ? 1 : 0;
12 }
```

Análise:

- a) O loop percorre os 32 bits de `bits` (assumindo 32-bit).
- b) Para cada bit, verifica se o bit *mais significativo* atual é 1. Se for, incrementa `total`.
- c) Após o loop, `bits` foi totalmente deslocado para a esquerda.
- d) `total` agora contém o *número de bits 1* (chamado *population count* ou *Hamming weight*).
- e) Retorna 1 se `total % 2 == 0` (par), ou 0 caso contrário.

Conclusão: `mystery` verifica a **paridade par** da contagem de bits 1. Retorna 1 (verdadeiro) se há um número **par** de bits 1; retorna 0 se ímpar.

Aplicação: verificação de paridade é usada em transmissão de dados e detecção de erros. Memórias ECC, RS-232, e protocolos antigos de telecomunicações usam essa ideia para detectar erros de transmissão.